TEST TRAP

Cybersecurity Analysis & Awareness Platform

PROJECT DOCUMENTATION

Final-Year Project | B.Tech Cybersecurity

Developed by

MOHAMMED KAIF	236F1A6219
E. NIKETHA REDDY	236F1A6213
SAI GANESH	246F1A6206
ADITHIYA	236F1A6204

Theme: Black & Crimson Red

Version 1.0 | November 2026

Table of Contents

1. Abstract

2. Introduction

3. Problem Statement

4. Objectives

5. Scope

6. Proposed System & Features

7. System Architecture

8. Technology Stack

9. Module Descriptions

10. System Workflow

11. Functional Requirements

12. Non-Functional Requirements

13. Security & Privacy Considerations

14. Testing Plan

15. Limitations

16. Future Enhancements

17. Conclusion

18. Team Details

1. Abstract

Test Trap is a unified cybersecurity analysis and awareness platform designed to help users inspect potentially risky digital content. It brings together source-code scanning, suspicious URL analysis, scam-message checking, and user-created Cyber Alerts in one web application. The platform aims to make common security checks accessible through a clear interface and understandable findings.

The project uses a React-based frontend, a Node.js and Express backend for scanning APIs, and Supabase for authentication and alert data storage. Test Trap is intended as an educational and awareness tool; its results are indicators for review, not guarantees of safety or proof of malicious activity.

2. Introduction

Digital users regularly encounter suspicious links, fraudulent messages, and insecure code. Understanding these risks can be difficult without security knowledge or specialized tools. Test Trap provides a single interface for basic analysis and reporting, with a focus on clarity, usability, and awareness.

The application follows a black and crimson visual theme and organizes its capabilities into four main tools.

3. Problem Statement

Security checks are often fragmented across different tools and require users to interpret technical output. Scam messages and phishing links can also rely on urgency, impersonation, or deceptive wording. Test Trap addresses this usability gap by consolidating selected checks and presenting findings in a structured format.

4. Objectives

• Provide a simple interface for scanning source code for selected insecure patterns and exposed secrets.

• Analyze URLs using structural checks and phishing indicators.

• Review messages for scam-related language and social-engineering patterns.

• Allow users to create, store, export, and share security alert reports.

• Present findings with understandable severity or verdict labels.

• Support user awareness while clearly communicating limitations.

5. Scope

Included

• Web-based user interface and authenticated workspace.

• Rule-based source-code and message analysis (as implemented).

• URL analysis using configured checks and indicators.

• User-initiated Cyber Alert creation and report sharing.

• Supabase-backed authentication and alert persistence.

Out of scope / not guaranteed

• No guarantee that every vulnerability, scam, or phishing URL will be detected.

• Not a replacement for professional penetration testing, antivirus, threat intelligence, or secure code review.

• Cyber Alerts are user-submitted reports and are not independently verified by the platform.

• Automated real-time threat notifications are not implied by the manual Cyber Alert feature.

6. Proposed System & Features

6.1 Code Scanner

The Code Scanner accepts source code for inspection and identifies selected insecure coding patterns, potential exposed secrets, or other configured indicators. It is intended to support an initial review and learning. Findings should be validated by a developer or security reviewer.

6.2 URL Scanner

The URL Scanner accepts a URL and evaluates configured URL characteristics and phishing indicators. Depending on the implementation, checks may include suspicious URL structure, IP-based hosts, unusual schemes, deceptive domain patterns, or other rule-based signals. A suspicious result is a warning for further verification—not a definitive determination.

6.3 Message Checker

The Message Checker accepts text, including English, Hindi, and Hinglish messages where supported by the rules. It looks for patterns associated with urgency, account threats, OTP/password requests, KYC pressure, suspicious links, prizes, and related social-engineering signals. It calculates a score and maps it to a verdict.

Configured verdict thresholds: score ≥ 60 → LIKELY_FRAUD; score ≥ 25 and < 60 → SUSPICIOUS; score < 25 → LIKELY_GENUINE. These labels are heuristic and can produce false positives or false negatives.

6.4 Cyber Alert

Cyber Alert is a manual report-generation feature. A user can enter a title, category, severity, description, and evidence; save the alert; export it as a PDF; and share a summary through WhatsApp. The platform stores alerts associated with the authenticated user. Alert content is user-provided and should not be treated as verified intelligence.

7. System Architecture

The application follows a client–server architecture:

• Frontend: React + Vite provides pages, forms, navigation, and result presentation.

• Backend: Node.js + Express exposes scanner endpoints and executes analysis logic.

• Authentication & data: Supabase Auth manages sessions; Supabase database stores Cyber Alert records with row-level security.

• Browser-to-backend communication: frontend sends JSON requests to API endpoints and displays returned analysis.

• Report export: PDF is generated in the browser using jsPDF; sharing uses a WhatsApp share URL.

Browser / React UI
Dashboard • Code Scanner • URL Scanner • Message Checker • Cyber Alert
↓ HTTPS / JSON API (local development may use HTTP)

Node.js + Express API
Scanner routes • Analysis services • Validation
↔
Supabase
Authentication • PostgreSQL • Row-Level Security

8. Technology Stack

Layer	Technology	Purpose
Frontend	React, Vite, React Router	UI, development server, page navigation
Styling	CSS, Tailwind utility classes (where used)	Responsive layout and visual design
Icons / visuals	Lucide React; Three.js / React Three Fiber components (where used)	Icons and dashboard visual elements
Backend	Node.js, Express	API and request handling
Database / Auth	Supabase, PostgreSQL, Supabase Auth	Authentication and alert storage
PDF export	jsPDF	Client-side alert PDF generation
Version control	Git / GitHub (if used by team)	Source management and collaboration

9. Module Descriptions

9.1 Authentication

Signup and login provide account access through Supabase Auth. Protected routes require an active session. Logout terminates the session and should prevent access to protected pages until the user signs in again.

9.2 Dashboard

The Dashboard acts as the authenticated workspace and links to the four security tools. It may display user profile information and tool navigation.

9.3 API routes (current integration)

The backend mounts scanner endpoints under /api/scanner, URL analysis under /api/url-scanner, and message analysis under /api/message-scanner. Exact request and response fields should be documented alongside the final implementation and verified through API testing.

9.4 Data storage

Cyber Alert records include a user association, title, category, severity, description, optional evidence, and timestamp. Row-Level Security policies should restrict reading, creating, and deleting records to their owner.

10. System Workflow

• User opens Test Trap and registers or logs in.

• Supabase Auth establishes the user session.

• User enters the Dashboard and selects a security tool.

• Frontend validates input and submits it to the relevant scanner API or Supabase operation.

• Backend or client-side analysis evaluates configured rules and returns findings.

• Frontend presents the verdict, score, and findings.

• User may manually create a Cyber Alert, save it, export a PDF, or share a summary.

• User logs out; protected routes require authentication again.

11. Functional Requirements

• FR-01: Users can register and log in.

• FR-02: Authenticated users can access the Dashboard and protected tools.

• FR-03: Users can submit source code to the Code Scanner.

• FR-04: Users can submit URLs to the URL Scanner.

• FR-05: Users can submit message text to the Message Checker.

• FR-06: Analysis results include a verdict and/or findings according to the module.

• FR-07: Users can create and manage their own Cyber Alerts.

• FR-08: Users can export alert reports as PDF and share summaries.

• FR-09: Users can log out and protected pages enforce authentication.

12. Non-Functional Requirements

• Usability: provide readable findings, clear validation, and consistent navigation.

• Reliability: handle invalid inputs and API failures without crashing the interface.

• Security: protect authenticated routes, validate inputs, and enforce database access policies.

• Maintainability: separate pages, routes, and analysis services into modular files.

• Performance: keep routine analysis responsive for reasonable input sizes.

• Compatibility: support modern desktop browsers; responsive behavior should be verified on mobile.

13. Security & Privacy Considerations

• Do not commit Supabase service-role keys, passwords, API secrets, or private credentials to source control or frontend code.

• Only the Supabase publishable/anon key intended for browser use may be placed in frontend environment configuration; enforce RLS for protected data.

• Never upload or scan code/messages/URLs containing real passwords, OTPs, personal data, or confidential business information during demonstrations.

• Use HTTPS in deployment and secure environment variables on the server.

• Validate and limit request sizes; handle errors safely and avoid logging sensitive user content.

• Restrict Cyber Alert database access using owner-based RLS policies and test them with multiple accounts.

• Scanner output is heuristic. Avoid presenting a 'safe' result as proof that content is harmless.

• Cyber Alert evidence may contain personal or confidential details; share only with consent and redact sensitive data.

14. Testing Plan

ID	Test	Expected result
T-01	Valid signup	Account created and session established
T-02	Invalid login	Clear error; no unauthorized access
T-03	Open each tool while logged in	Correct tool page loads
T-04	Open protected route while logged out	Redirect to login
T-05	Code Scanner safe sample	Result returned without crash
T-06	Code Scanner insecure sample	Configured finding detected
T-07	URL Scanner valid URL	Analysis result returned
T-08	URL Scanner malformed input	Validation message; no crash
T-09	Message Checker benign text	Low score / likely genuine heuristic
T-10	Message Checker scam-like text	Suspicious signals and higher score
T-11	Create Cyber Alert	Record saved for current user
T-12	Export alert PDF	PDF generated with entered report data
T-13	Delete alert	Record removed from user's list
T-14	Cross-user alert access	Other user's records denied by RLS
T-15	Logout then refresh protected route	Login required
T-16	API unavailable	Friendly error displayed

Testing should be performed with synthetic examples and accounts controlled by the project team. Record actual pass/fail outcomes separately; this document lists planned tests and does not claim they have all passed.

15. Limitations

• Rule-based detectors can miss new or obfuscated threats and may flag benign content.

• URL reputation checks depend on configured sources and may not reflect real-time reputation unless explicitly integrated.

• Message language coverage depends on implemented rules; multilingual text may be inconsistently detected.

• Source-code scanning is not a full static-analysis engine unless additional parsers and rules are implemented.

• Cyber Alert reports are user-created and not automatically validated.

• Internet access, backend availability, and Supabase configuration affect application operation.

16. Future Enhancements

• Add configurable severity explanations and remediation guidance.

• Expand code scanning with language-specific parsers and more robust secret detection.

• Add safe, reputable URL reputation integrations and transparent source attribution.

• Improve Hindi/Hinglish message rules with carefully evaluated datasets.

• Add scan history and dashboard statistics backed by real database records.

• Add administrator moderation/verification workflow for shared alerts, with privacy safeguards.

• Improve accessibility, responsive layout, and automated unit/integration testing.

• Add CI checks, dependency scanning, and deployment hardening.

17. Conclusion

Test Trap consolidates selected cybersecurity checks and manual alert reporting into a single accessible web platform. Its modular design supports future improvements in detection quality, usability, and security controls. The project is best positioned as an educational security-awareness and preliminary analysis tool, with clear limitations and responsible use.

18. Team Details

S. No.	Name	Student ID
1	MOHAMMED KAIF	236F1A6219
2	E. NIKETHA REDDY	236F1A6213
3	SAI GANESH	246F1A6206
4	ADITHIYA	236F1A6204

Project: Test Trap | Project developers listed above. No passwords, private keys, personal contact details, or confidential credentials are included.